МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
“КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ ІМЕНІ ІГОРЯ СІКОРСЬКОГО”

Факультет прикладної математики
Кафедра програмного забезпечення комп’ютерних систем

Лабораторна робота № 2

з дисципліни “ Прикладне програмне забезпечення мовою Python”
тема “Робота з ітераторами та генераторами”

Виконав
Студент III курсу
групи КП-31
Власенко Артем

Київ 2026

Варіант 2

1. Написати функцію, яка приймає три цілі числа *x*, *a*, *b* і за допомогою генераторного виразу створює і повертає новий список довжиною *x* для випадкових чисел від *a* до *b*. Для рішення даного завдання рекомендується використовувати функцію *random.randint()*.

2. Написати функцію, яка приймає цілочисельний список, що складається з *n* елементів, та за допомогою генераторного вирази створює та повертає список, що містить тільки додатні елементи вхідного список.

3. Написати функцію, яка приймає список та за допомогою генераторного вирази створює і повертає словник, де як ключі будуть номери позицій елементів вхідного словника, а значеннями – самі елементи.

4. Написати функцію, яка приймає список дійсних чисел [1.3, 2.5, 3.1]. Створіть множину квадратів цих чисел, використавши для цього генератор множин. Виведіть отриману множину на екран.

5. Числа Фібоначчі представляють послідовність, одержувану в результаті додавання двох попередніх елементів. Починається колекція з чисел: 1, 1, 2, 3, 5… Вона досить швидко зростає, тому обчислення великих значень займає чимало часу. Створіть функцію fib(n), що генерує n чисел Фібоначчі з мінімальними витратами ресурсів. Для реалізації цієї функції потрібно звернутися до інструкції yield. Вона не зберігає в оперативній пам'яті велику послідовність, а дає можливість виводити проміжні результати по одному.

```python
import random


def task1(x, a, b):
    if a > b:
        a, b = b, a
    return [random.randint(a, b) for _ in range(x)]


def task2(lst):
    return [x for x in lst if x > 0]


def task3(lst):
    return {i: v for i, v in enumerate(lst)}


def task4(lst):
    return {x ** 2 for x in lst}


def task5(n):
    a, b = 1, 1
    for _ in range(n):
        yield a
        a, b = b, a + b
```

```python
if __name__ == "__main__":
    # Task 1
    print(task1(5, 1, 10))    # [3, 7, 1, 9, 4]
    print(task1(3, 5, 5))     # [5, 5, 5]

    # Task 2
    print(task2([-3, 0, 5, -1, 7, 2, -9, 4]))  # [5, 7, 2, 4]
    print(task2([-1, -2, -3]))                 # []

    # Task 3
    print(task3(["alpha", "beta", "gamma"]))    # {0: 'alpha', 1: 'beta', 2: 'gamma'}
    print(task3([]))                            # {}

    # Task 4
    print(task4([1.3, 2.5, 3.1]))  # {1.690000000000002, 6.25, 9.610000000000001}
    print(task4([2, 3]))           # {4, 9}

    # Task 5
    print(list(task5(10)))  # [1, 1, 2, 3, 5, 8, 13, 21, 34, 55]
    print(list(task5(1)))   # [1]
```

Висновки

Виконуючи цю роботу були розширенні знання про можливості мови програмування Python.

Питання для самоперевірки

1. Ітератор - це об’єкт, який дозволяє пройтися по набору даних, один елемент за одним, без необхідності зберігати усю колекцію в пам'яті. Він надає можливість послідовно отримувати наступні елементи об'єкта під час ітерації. У Python, ітератори зазвичай реалізовані через методи iter та next.
2. Ітерабельний об'єкт - це будь-який об'єкт в Python, через який можна пройтися за допомогою циклу for. Це може бути список, кортеж, рядок, множина, словник, файловий об'єкт тощо.
3. Ітерабельний об'єкт забезпечує можливість ітерації, тоді як ітератор - це сам процес ітерації, що в змозі по одному повертати свої елементи. Кожен ітератор є ітерабельним об'єктом, але не кожен ітерабельний об'єкт є ітератором.
4. Для створення ітератора потрібно визначити клас, що містить методи iter() та next(). Метод iter() повертає сам ітератор, в той час як метод next() видає наступний елемент послідовності або породжує виняток StopIteration, якщо елементи закінчились.
5. Функція iter() створює ітератор з об'єкта, а next() використовується для отримання наступного елемента з ітератора. При вичерпанні всіх елементів next() породжує виняток StopIteration, який можна обробити або ігнорувати за допомогою параметра default.
6. Генератор - це спеціальний вид функції в Python, що дозволяє зручно створювати ітератори за допомогою зручного синтаксису, відомого як генераторні вирази або генераторні функції. Головна особливість генераторів полягає в тому, що вони генерують значення по одному та відкладають обчислення до моменту потреби.
7. Генератори створюють послідовність значень за запитом, тоді як ітератори дозволяють ітерувати через послідовність, яка вже зберігається в пам'яті.

8. Генератори можна створювати за допомогою функцій, що містять вираз yield. Також можна використовувати генераторні вирази або декоратор @staticmethod.
9. Коли функція знаходить вираз yield, вона призупиняє своє виконання та повертає значення, зазначене після yield. Функція залишається у "замороженому" стані, зберігаючи контекст виконання, такий як значення локальних змінних та точку виконання коду. При наступному виклику генератора він продовжує виконання з місця, де зупинився, та генерує значення відповідно до наступного виразу yield.
10. Оператор return використовується в звичайних функціях для повернення значення та завершення виконання функції. Оператор yield в генераторах використовується для повернення значення та тимчасової призупинки виконання функції, зберігаючи її поточний стан.
11. Метод send() - це метод генератора, що дозволяє передавати значення до генератора з зовнішнього коду, яке може бути оброблене як yield у тілі генератора. Метод throw() - це метод генератора, що дозволяє викидати виключення у генераторі з зовнішнього коду, що породжує виняток там, де було останнє викликане yield у генераторі. Метод close() - це метод генератора, який дозволяє закрити генератор, який більше не буде використовуватися, та будь-який наступний виклик генератора спричинить виняток StopIteration.
12. Нескінченний генератор - це генератор, який продовжує генерувати значення без обмежень. Це можна здійснити, використовуючи цикл while або True в тілі генератора, а також оператор yield для повернення значень.